

GLEAM

—



| — | — — —



Introduction

■ Who even is this guy?

- Software Engineer @iHeart
- Keyboard and ergonomics enthusiast
- Die hard terminal fanboy
- Loves typesystems and statically typed languages (Yes... Even Typescript)

Here is me with me cat, Fritzzy =3



What is Gleam?

From `gleam.run` (<https://gleam.run/>):

Gleam is a friendly language for building type-safe systems that scale!

■ Power of Functional Programming

Gleam is a purely functional language. Immutable data, powerful type system, simple syntax combine to give a developer experience that sparks joy.

■ Batteries Included

Gleam comes with many great built in tools to handle all stages of development.

- Language Server
- Formatter
- Editor Integrations
- Package Manager

■ Helpful Error Messages

The Gleam compiler aims to assist you with any errors. Gone are the days of the cryptic 10 page long Typescript errors.

■ Flexible Runtime

You can compile Gleam for one of two targets:

- **Erlang**
 - Leveraging the power of the BEAM VM, the Erlang target can make use of the **Open Telephony Protocol** (or **OTP**) to handle massively concurrent systems that fail gracefully.
- **Javascript**
 - For when you want to run your Gleam code inside a browser, or even interop with JS/TS packages. The Gleam compiler will even generate Typescript type definitions for your code.

Functional Programming 101

■ Declarative Design

Functional languages are **declarative**. They aim to describe what should happen, not necessarily how that happens[1].

■ Composition and Higher Order Functions

In a functional language, functions can take other functions as parameters. As such, your logic is a series of compositions of functions, rather than lists of instructions.

■ Immutable Data

Bindings in functional languages are **immutable**. This is just a fancy way of saying their data cannot change.

■ Pure Functions

Functions cannot have **side effects**[2]. Meaning for any given **input** a function will always return the same **output**.

1. There is definitely overlap here, especially in multi-paradigm languages, such as Typescript.
2. Modern functional programs have mechanisms to interact with side effects but that's a little out of scope here.

Modules

Gleam code is split into modules. Each module is comprised of functions, types, constants, etc. Each module relates to a specific file in the `src` directory of a gleam project. For instance, let's say you have file called `world.gleam` inside of a directory called `hello/` in your `src` directory.

When writing definitions you can use the `pub` keyword to specify what you want to be available in other modules. Definitions not prefixed with `pub` are only available from within that module.

This would be similar to the following Typescript

```
export function my_function() {  
  //...  
}  
  
function module_function() {  
  //...  
}
```

```
// Will be available outside of this module  
pub fn my_function() {  
  todo  
}  
  
// Only available within this module  
fn module_function() {  
  todo  
}
```

Imports

To import a module you would use the `import` keyword inside of another module.

Gleam

```
import hello/world
```

Typescript

```
import world from "hello/world"
```

You can also alias your imports using the `as` keyword if you want to use a different name.

Gleam

```
import hello/world as wrld
```

Typescript

```
import wrld from "hello/world"
```

Alternatively you can import specific parts of a module using `unqualified imports` using the `.{name}` syntax.

Gleam

```
import hello/world.{my_function}
```

Typescript

```
import {my_function} from "hello/world"
```

Values and Assignments

In Gleam the concept of variables doesn't really exist, as values are immutable. However, you can still assign values to identifiers for easy reuse within your functions:

```
let a = 4
```

Identifiers in Gleam are written using snake_case. So where in Typescript it is entirely up to the developers (And people seem to generally prefer camelCase or PascalCase), this name convention is **enforced** by the Gleam compiler.

```
// Invalid, will get a compiler error
let helloWorld = "Hello, World!"
// Correct
let hello_world = "Hello, World!"
```

Like Typescript, type annotations are also optional in Gleam:

```
let a = "a"
// is the same as
let a: String = "a"
```

Functions

Functions are the core of Gleam programs. You use the `fn` keyword to define a function in Gleam. Unlike in Typescript, type annotations on function parameters are optional in Gleam. This is because the type system is able to look at how the parameters are used to infer their types. However, it is still considered good practice to include type parameters on functions. You use the `->` syntax to specify a return type:

```
fn double(a: Int) -> Int {  
  a * 2  
}  
// Is the same as  
fn double(a) {  
  a * 2  
}
```

Another thing to notice is that Gleam uses implicit returns. Whatever the last expression in a block is will be returned.

Gleam also has `anonymous functions`. These are very helpful when you are passing a function into another function as a value:

```
higher_order_function(fn(x) {x + 1})
```


Pipelines and Function Captures

One of the most powerful features of functional languages is the ability to create **pipelines** of functions, where the output of the previous function becomes the input for the next. Gleam uses the `|>` operator for this. For example, if we wanted to create a pipeline that used a **double** function into a **sqrt** function:

```
let doubled_sqrt = double(4) |> sqrt
// The same as sqrt(double(4))
```

When you combine this with **function captures** which allow you to take a function, give it some arguments, and get back another function that takes a subset of arguments that you can call later. For instance, let's say we had a **pow(num, power)** function that raises a number to a power. We could use a function capture to turn that into a **cube** function to reuse later:

```
let cube = pow(_, 3)
let two_cubed = cube(2) // Same as pow(2, 3)
let five_cubed = cube(5) // Same as pow(5, 3)
```

Flow Control

Handling branching conditions is vital for software. In Typescript you would use features like `if/else` or `switch` statements. In Gleam you get a supercharged version of `switch` statements called **Pattern Matching**. You use the `case` keyword to create a pattern match, and inside you must handle all possible branches.

```
fn is_a_or_z(str: String) -> Bool {  
  case str {  
    "a" | "z" -> True  
    _ -> False  
  }  
}
```

In typescript this might look something like:

```
function is_a_or_z(str: string): boolean {  
  if (str === "a" || str === "z") {  
    return true;  
  }  
  return false;  
}
```

Recursion

Gleam also doesn't include `loops` as there is no way to modify values to handle looping conditions. As such we use `recursion`. This is simply a functional calling itself until a base case is met. For instance if we wanted to loop through all the numbers from 0-5 and print them in Typescript you might do:

```
function printTilFive() {  
  for(let i = 0; i <= 5; i++) {  
    console.log(i)  
  }  
}
```

In Gleam you would need to use recursion for this:

```
fn print_til_five(val: Int) -> Nil {  
  case val {  
    val if val <= 5 && val >= 0 -> {  
      echo val  
      print_til_five(val + 1)  
    }  
    _ -> Nil  
  }  
}
```

Simple Types

Gleam ships with a number of "simple" types. These include:

- `String`
- `Int`
- `Float`
- `Bool`

This is pretty similar to Typescript, except that instead of a unified `number` type, our numbers are split into the `Int` and `Float` types. This can be simplified down to whole numbers vs rational numbers.

Each of these primitive types also has their own `module` included in the standard library (or `stdlib`). These give you all of the utility functions that might exist on the prototype for the value, like in Typescript.

```
import gleam/string
let str = "the quick brown fox jumped over the
lazy dog"
let words = str |> string.split(" ")
```

```
let str = "the quick brown fox jumped over the
lazy dog"
let words = str.split(" ")
```

Type parameters

--> Just like in Typescript, Gleam supports type parameters, aka **generics**. They are specified using `()` after a type that accepts generics.

Typescript

```
type MyList<T> = T[]
```

Gleam

```
type MyList(T) = List(T)
```

Unlike in Typescript, the Gleam compiler is smart enough to know when a type in a function declaration is a type parameter without any special syntax

Typescript

```
function generic<T>(t: T) {  
  //...  
}
```

Gleam

```
fn generic(t: T) {  
  //...  
}
```

List Type

Lists are basically Typescript arrays. They use a very similar syntax as well. However, in Gleam all elements of the list must be the same type and you cannot index into a `List` type directly (since they are implemented with linked lists).

Typescript

```
let arr = ["a", "b", "c"]
```

Gleam

```
let arr = ["a", "b", "c"]
```

Due to Gleam's declarative nature, some things you may be used to with arrays aren't really possible, or aren't quite as straightforward. For instance, indexing into a `List` in Gleam runs in $O(n)$ time, because it has to traverse the linked list.

Another example is appending to the end of a list. This would require a **full** traversal of the List, so it is recommended to prepend to the beginning of the list using the `[item, ..list]` syntax.

Tuple Type

Tuples work similarly to tuples in Typescript, but they are their own data type in Gleam. In Typescript they are implemented using arrays. The Typescript compiler doesn't infer tuples very well (With the exception of using the `const` type assertion).

You can use `#(...)` to instantiate a tuple in Gleam. You can also destructure the contents of a tuple similar to in Typescript.

Typescript

```
// type: string[]
let notTup = ["a", "b", "c"]
// type: [string, string, string]
let tup: [string, string, string] = ["a", "b", "c"];
// type: ["a", "b", "c"]
let constTup = ["a", "b", "c"] as const;
// Destructure any number of tuple members
let [a, _, c] = tup;
```

Gleam

```
let tup = #("a", "b", "c")
// Destructure any number of tuple
members
let #(a, _, c) = tup;
```

Dict Type

Dicts (short for **dictionary**) are similar to objects, or more aptly the ES6 **Map**. They are a hashmap of key:value pairs. Unlike in Typescript though, all the types in the Dict need to be the same type. Since Gleam uses `{}` to denote **blocks** of expressions, you can't use them to instantiate a **Dict**.

Typescript

```
// type: {a: string; b: string}
let obj = {
  "a": "letter a",
  "b": "letter b"
}
// type Map<string, string>
let map = new Map(obj)
```

Gleam

```
import gleam/dict
let obj = dict.new()
  |> dict.insert("a", "letter a")
  |> dict.insert("b", "letter b")
```

NOTE: Unlike the **List** or **Tuple** type, you need to import the **Dict** type from the `stdlib` to use it.

Custom Types

Custom types in Gleam look similar to `enums` in Typescript. However, the `variants` of the type can also include additional fields, similar to properties on an object. This is especially nice when you're representing tagged unions. Lets say we have a `Message` type.

Typescript

```
enum MessageType {  
  Incoming  
  Outgoing  
}  
  
type Message = {  
  messageType: MessageType.Incoming;  
  from: string;  
  message: string;  
} | {  
  messageType: MessageType.Outgoing;  
  to: string;  
  message: string;  
}
```

Gleam

```
type Message {  
  Incoming(  
    message: String,  
    from: String  
  ),  
  Outgoing(  
    message: String,  
    to: String  
  )  
}
```

Option Type

There is not really a concept of `null` or `undefined` in Gleam. So unlike in Typescript where if you had an optional item you might use a type such as:

```
let optionalString: string | undefined;
```

In Gleam you would use the `Option` type. This type has two variants:

 Gleam

```
type Option(T) {  
  Some(T)  
  None  
}
```

 Typescript

```
type Option<T> = {  
  _tag: "some";  
  value: T;  
} | { _tag: "none" }
```

NOTE: You need to import the `Option` type from the `stdlib` to use it.

Result Type

Handling errors is pretty different than many non-functional languages. Errors in Gleam are treated as values, not exceptions. This makes it much easier to handle them in a predictable manner. Where in Typescript you may use the `throw` keyword, such as

```
() => {  
  throw new Error("Oh no!")  
}
```

While this is pretty simple the big downside is that you lose type safety. With Gleam you would use the `Result` type.

Gleam

```
type Result(Val, Err) {  
  Ok(Val)  
  Error(Err)  
}
```

TypeScript

```
type Result<Val, Err> = {  
  _tag: "ok";  
  value: Val;  
} | {  
  _tag: "error";  
  value: Err;  
}
```

Example

Let's take a look at a simple Todo App written in Gleam. The code can be found on github at:



Why Gleam

I have three main cases for why you should consider Gleam for your next project:

■ The FP Noob

You want to learn more about functional programming but don't want to step into the world of monads and endofunctors ala Haskell.

■ The Beam Enjoyer

You want to make use of the BEAM VM and the existing ecosystem of Erlang and Elixir packages, but you prefer the simplicity and style of C or Javascript.

■ Any sane person

Lucy (The Gleam mascot) is just cute af.

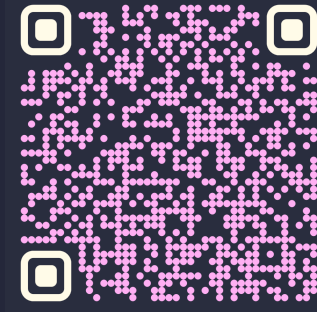


Additional Resources

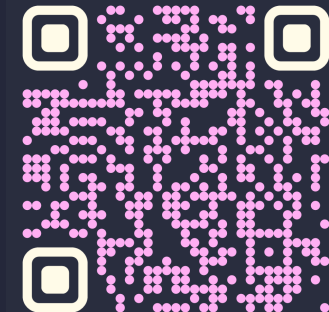
■ Gleam Language Tour



■ Install



■ Project Tutorial



Fin

Thank
you!



